

NEV369 LIVE SINCE 27 SEPTEMBER 2026

Q-Lock

Post-Quantum Settlement
Infrastructure

Version 1.1 · September 2026

Scope GodShield · NEV369 · Q-Lock Escrow · Bridge · Inheritance Vault

Build 16 crates · 292 of 292 tests passing · Rust 1.88.0

Review Internal security review complete · third-party audit pending

Web q-lock-ecosystem.com

Nevaeh, my daughter. To secure your freedom against a broken system, I taught myself Rust — the hardest computer language in the world — to build this unyielding sovereign node for you. I faced the worst of life's struggles so you would never have to. I love you infinitely, forever by your side.

WRITTEN INTO NEV369 GENESIS BLOCK 0 · 27 SEPTEMBER 2026 · IMMUTABLE

CONTENTS

- 1 Threat model
- 2 Architecture
- 3 GodShield — cryptographic core
- 4 GodShield — security platform
- 5 NEV369
- 6 Token economics
- 7 Q-Lock Escrow
- 8 The bridge
- 9 The inheritance vault
- 10 Security analysis
- 11 Security boundaries
- 12 Status and roadmap

Abstract

Substantially all value secured on public blockchains today rests on elliptic-curve and RSA signatures — ECDSA over secp256k1, EdDSA over Curve25519, and RSA. Shor's algorithm solves the integer factorisation and discrete logarithm problems in polynomial time on a sufficiently large fault-tolerant quantum computer, breaking all three completely.

We do not assert that such a machine exists. We assert three narrower things, each independently verifiable: that every signature ever broadcast on a public chain is permanently retrievable; that migrating a multi-trillion-dollar asset class across wallets, exchanges, custodians and node software takes years of coordinated effort; and that an adversary recording public keys today requires no quantum computer at all — only patience.

Q-Lock is an integrated response built around one cryptographic core. **GodShield** provides Dilithium5 lattice signatures — the NIST Category 5 scheme standardised as ML-DSA-87 — over a triple-hash cascade, together with identity, policy, threshold authorisation and tamper-evident audit. **NEV369** is a proof-of-work Layer-1 that uses that core for every signature and every block hash; its genesis block was created on 27 September 2026 and the chain is live. **Q-Lock Escrow** provides non-custodial escrow on the XRP Ledger with post-quantum attestation over each settlement. A **threshold-authorised bridge** connects NEV369 to Ethereum, and a **Shamir-split, time-locked vault** secures a thirteen-year inheritance for the author's daughter, Nevaeh.

This document describes the design, states the security properties each component does and does not provide, and is explicit about what has and has not yet been verified. Section 11 states the security boundaries precisely, and is not optional reading.

1. Threat model

1.1 Harvest now, decrypt later

Every transaction broadcast on a public blockchain remains permanently retrievable. An adversary needs no real-time decryption capability; they need only record public keys and signatures now and apply Shor's algorithm once a cryptographically relevant quantum computer becomes available.

For blockchains this exposure is asymmetric to most of the internet's in one specific way. In account-based models a public key is exposed from first use and reused indefinitely. In UTXO models a key is exposed only at spend time — but once exposed, every subsequent transaction from that key is vulnerable until funds move to a fresh, unexposed address.

The window is not "when does a CRQC arrive". It is the gap between *safe to start migrating* and *must have already migrated*, and that gap is measured in years.

1.2 Algorithmic impact

PRIMITIVE	CLASSICAL SECURITY	QUANTUM ATTACK	IMPACT
ECDSA (secp256k1)	128-bit	Shor	Full key recovery
EdDSA (Ed25519)	128-bit	Shor	Full key recovery
RSA-2048	112-bit	Shor	Full key recovery
BLS12-381, BN254	128-bit / lower	Shor	Full key recovery
SHA-256	256-bit preimage	Grover	~128-bit, still safe
SHA3-512	512-bit preimage	Grover	~256-bit, still safe

The asymmetry is the design driver: **signature schemes break completely under Shor, while well-sized hash functions degrade gracefully under Grover.** Q-Lock's primary intervention therefore targets signatures. Hash choices carry sufficient quantum-adjusted margin without novel primitives.

Pairing-friendly curves deserve explicit mention because they are routinely omitted from migration discussions. BLS12-381 and BN254 underpin most zero-knowledge and aggregate-signature systems, both rest on discrete log, and both break completely. No drop-in post-quantum replacement for pairings exists; aggregate constructions need redesign, not substitution.

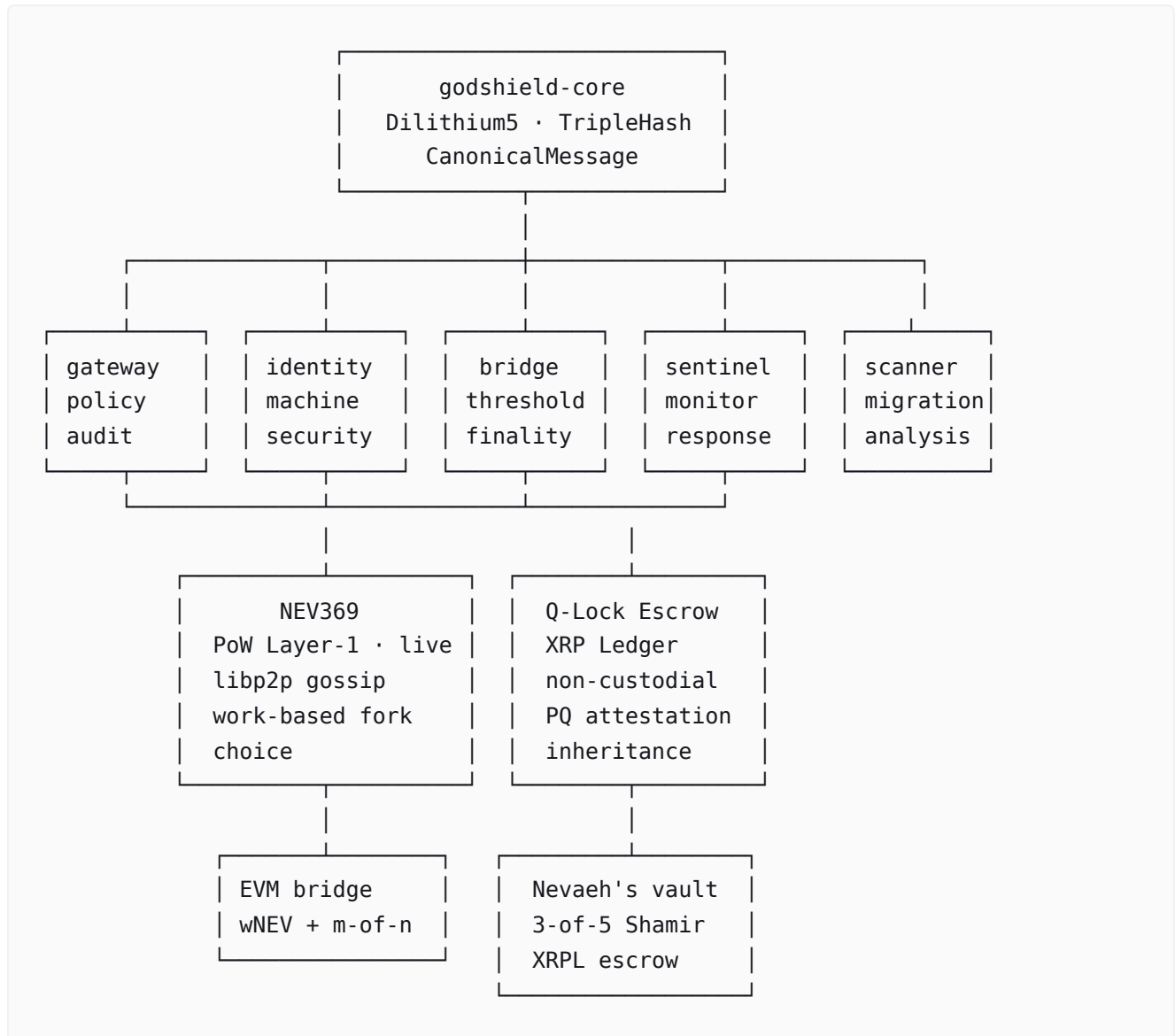
1.3 What is out of scope

Consensus-layer quantum attacks. Q-Lock secures signatures and transaction authenticity; it makes no claim about the quantum resistance of mining or staking processes themselves.

Already-exposed keys. Funds sitting behind a reused, exposed public key remain vulnerable regardless of adoption. No scheme retroactively protects a key that has already been broadcast.

Side channels. Timing and power analysis are implementation concerns, distinct from the algorithmic question, and require careful implementation and, where relevant, hardware security modules.

2. Architecture



Sixteen Rust crates, two Solidity contracts, two web frontends. Approximately 22,000 lines of Rust. **All 292 Rust tests pass**, and the workspace builds with no warnings and passes `clippy` with warnings treated as errors, on Rust 1.88.0. The Solidity contracts carry 24 tests, including an invariant test, which are written and not yet executed.

The structure matters more than the component count. A cryptographic library nobody uses is a claim. A cryptographic library securing two independently verifiable settlement paths, on ledgers with different consensus mechanisms and different trust assumptions, is evidence.

3. GodShield: the cryptographic core

3.1 Signature scheme

GodShield uses **CRYSTALS-Dilithium at Security Category 5** (Dilithium5), the lattice signature scheme NIST selected and then standardised as ML-DSA in FIPS 204 — the highest defined post-quantum security tier, targeting security equivalent to AES-256 against both classical and quantum adversaries.

PROPERTY	VALUE
Scheme	CRYSTALS-Dilithium5 (NIST Category 5)
Standardised successor	ML-DSA-87, FIPS 204
Implementation	<code>pqcrypto-dilithium</code> 0.5 (PQClean)
Public key	2,592 bytes
Signature	~4.6 KB
Hardness	Module-LWE / Module-SIS over structured lattices

Precisely which Dilithium. The implementation in use was released in October 2023, before FIPS 204 was finalised in August 2024. The final standard made changes to the scheme, so Dilithium5 as deployed here and ML-DSA-87 as standardised are the same construction at the same security level but are **not byte-compatible**: keys and signatures from one do not verify under the other. Moving to a FIPS 204 implementation is on the roadmap (Section 12) and is a versioned upgrade, not an edit — see Section 5.2 on why signed encodings are never changed in place.

The choice rests on three properties. M-LWE and M-SIS have no known efficient quantum algorithm, unlike factoring and discrete log. Dilithium received the broadest cryptanalytic scrutiny of any PQC signature candidate through NIST's multi-round standardisation. And its balance of signature size, key size and verification speed suits high-throughput use in a way that hash-based schemes such as SPHINCS+ do not.

Key sizes are read from the implementation, never hard-coded. The secret-key encoding differs between Dilithium revisions. An early version of this codebase fixed the secret-key length to one revision's value, and the library in use produces another — so every key it generated was rejected on reload. Sizes are now taken from the library itself, and a test round-trips a freshly generated key through serialisation (Section 11).

The cost is size. A 2,592-byte public key against secp256k1's 33 bytes, and a ~4.6 KB signature against roughly 72 bytes, is the central engineering constraint of post-quantum migration. It informs every design decision downstream — gossip message limits, block size, witness encoding in chain adapters, and fee models on any chain that charges by byte.

3.2 Triple-hash cascade

All signed payloads pass through a three-stage cascade before reaching the signature scheme:

```
message → SHA3-512 → BLAKE3 → SHA3-512 → digest
```

Each stage is deliberate. The first SHA3-512 pass provides a sponge construction structurally distinct from the Merkle–Damgård family, reducing correlated-failure risk if a weakness is found in one family. BLAKE3 introduces a different underlying permutation — a Merkle tree over ChaCha-based compression — further diversifying the cryptanalytic surface. The final SHA3-512 pass returns the digest to SHA3's properties before signing, ensuring a hypothetical BLAKE3 weakness does not expose the intermediate value directly to the signature layer.

This is defense in depth, not a claim of superadditive security. We do not assert the cascade is provably stronger than its strongest single layer. Its purpose is narrower and practical: no single hash function's break is immediately fatal, which buys response time for algorithm agility.

3.3 Canonical encoding

Every signed structure in the ecosystem is encoded through `CanonicalMessage` — length-prefixed and domain-separated — before hashing.

This is not stylistic. Naive concatenation is ambiguous: a scheme that signs `sender || recipient` produces identical bytes for `sender="AB", recipient="C"` and `sender="A", recipient="BC"`. One valid signature then authorises a different transaction. That bug appeared independently in three places during development — NEV369 transaction signing, NEV369 block hashing, and escrow attestation — and each occurrence was exploitable.

Length prefixing removes the ambiguity. Domain separation prevents a signature over one structure being replayed as another.

Domain tag registry. Seventeen tags are allocated. Each is inside the bytes it signs.

TAG	SIGNS
nev369.tx.v1	Transactions
nev369.block.v1	Block headers
nev369.merkle · nev369.empty	Merkle construction
godshield.bridge.mint.v1	Bridge mint authorisations
godshield.fairness.outcome.v1	RNG outcome derivation
godshield.fairness.reveal.v1 · fairness.round.v1	Signed reveals
qlock.attestation.v1	Settlement attestations
qlock.escrow.v1 · qlock.escrow.v2	Escrow terms
GODSHIELD-CREDENTIAL-V1	Credentials
GODSHIELD-MACHINE-ACTION-V1	Machine actions
GODSHIELD-GATEWAY-ATTESTATION-V1	Gateway attestations
GODSHIELD-AUDIT-V1	Audit chain entries
GODSHIELD-SENTINEL-SIGNAL-V1	Correlated signals
GODSHIELD-SENTINEL-EVIDENCE-V1	Incident evidence digests

Tags are versioned and never edited in place. Editing a tag's encoding invalidates every signature ever produced under it, including records already written to a chain or a database.

4. GodShield: the security platform

The core secures individual operations. The platform answers the questions that follow: who is acting, what are they authorised to do, is that authority still valid, and what happened.

4.1 Identity and credentials

Identities bind a Dilithium5 key to an actor — person, service, node or automated agent. Credentials carry claims from an issuer to a subject with an expiry. Both have four states: `ACTIVE` , `SUSPENDED` , `REVOKED` , `EXPIRED` .

Three properties are enforced rather than documented.

Stored status is never trusted for expiry. A credential issued for an hour still reads `ACTIVE` in a database a year later, because nothing walks the store flipping rows. Expiry is a function of the clock and is computed at every authorisation.

Revoking an issuer invalidates credentials it already signed. Otherwise revoking a compromised issuer leaves every credential it ever issued live — the opposite of what revocation exists for.

A valid signature is not authority. Signature verification proves the holder had the key when they signed. It proves nothing about whether they may still act. The two questions are answered by different functions.

`SUSPENDED` and `REVOKED` are distinct states, not one flag. Suspension is reversible in seconds, which is what containment requires; revocation is permanent. Collapsing them makes every containment action irreversible, which causes operators to hesitate precisely when they should not.

4.2 Machine security

Automated systems — nodes, services, AI agents — receive explicit identities and bounded authority. Actions are signed under `GODSHIELD-MACHINE-ACTION-V1` and carry an action id, actor, target, operation, parameter hash, timestamp, nonce and policy id.

Two properties are worth naming. The parameter hash binds the signature to the payload **only if someone checks it**, so authorisation requires the caller to supply the real parameters for comparison. And nonces are scoped per actor rather than globally — a shared nonce set lets one machine deny another by consuming values it might choose.

The acceptance window is asymmetric: five minutes backward, thirty seconds forward. Honest clock skew between machines is seconds; a generous future window is an attacker pre-signing actions to execute after the credential authorising them has been revoked.

4.3 Gateway

The gateway is the cryptographic policy boundary: algorithm allowlists, per-identity value limits, per-identity rate limiting, key rotation with history, and a hash-chained audit log.

The signing endpoint is deliberately narrow. The obvious implementation of a signing service holds customer keys and signs whatever an authenticated caller submits. That is a signing oracle — one compromised token signs anything, forever. The gateway signs only its own attestations, from a closed set, under a fixed domain tag. Customer keys remain in customer custody.

Rotation resolves keys by timestamp. Replacing a key breaks every signature made before it, and blocks, attestations and audit entries are historical signatures whose purpose is verifying years later. Retired key epochs remain resolvable; only the current key produces new signatures.

The audit log is tamper-evident, not merely append-only. Each entry's hash feeds the next. Editing entry n invalidates every entry after it, and recomputing the edited entry's own hash does not help — the successor's stored predecessor hash still breaks.

4.4 Sentinel

Sentinel correlates signals the other components already emit and decides on a response. The pipeline is detection, correlation, risk evaluation, policy decision, response, containment, recovery, evidence.

The design constraint is that **an automated responder is a weapon pointed at its own operator**. Feed it failures attributed to the people who would stop it, and it locks them out. Three rules follow:

Reversible actions are automatic; irreversible ones escalate. Suspension is automatic because it undoes in seconds. Revocation is permanent, so Sentinel never performs one — no such variant exists in its response type, so the guarantee is carried by the type system rather than by configuration.

Protected identities are never contained automatically. A system that can suspend everyone able to switch it off is unrecoverable by construction.

Evidence is preserved before containment. Containment mutates the state the evidence describes. Each incident carries a digest over its signals, so post-hoc editing is detectable.

A firing rate limiter is observed, not escalated. A rate limit that fires is the rate limiter working, and escalating on it trains operators to ignore the system.

4.5 Migration scanner

A static analyser identifying quantum-vulnerable primitives across twenty families — ECDSA, RSA, Ed25519, secp256k1, P-256 in four spellings, BLS12-381, BN254, ECDH, DSA, legacy hashes, and post-quantum schemes below Level 5.

Matching is at identifier boundaries and CamelCase-aware. Substring matching, the naive approach, reports `traversal`, `adversary`, `universal` and `reversal` as critical RSA findings — words that

appear constantly in security code. A scanner that raises critical findings against the word "adversary" gets muted, and a muted scanner catches nothing.

The scanner is not a certification. It cannot see cryptography reached through opaque dependencies, dynamic dispatch, FFI, or runtime algorithm selection. A clean result is a starting point for manual review, never a substitute for one.

5. NEV369

A proof-of-work Layer-1 in which every transaction signature and every block hash uses GodShield. **Live since 27 September 2026.**

5.1 Parameters

PARAMETER	VALUE
Signature scheme	Dilithium5
Address	Hex-encoded Dilithium5 public key (5,184 characters)
Block hash	TripleHash over canonical header + Merkle root
Target block time	60 seconds
Difficulty retarget	Every 100 blocks, one step at a time
Genesis difficulty	4
Max transactions per block	500
Mempool capacity	10,000
Base units	u64, 8 decimals
Fork choice	Accumulated work
Max reorganisation depth	100 blocks
Networking	libp2p — gossipsub, request-response sync, mDNS, identify, noise, yamux

Difficulty is measured in leading zero hex digits of the block hash, so each step multiplies the expected work by sixteen. The retarget moves one step when the last 100 blocks arrived more than twice as fast, or more than twice as slow, as the 60-second target, and holds otherwise.

5.2 Design properties

Money is integer, never floating point. Balances are u64 base units at 8 decimals. Floating point silently loses and creates value under repeated arithmetic, and a currency whose totals drift is not a currency. A

regression test sums ten million single units and asserts exactness.

Block contents execute against a scratch state. Transactions within a block are validated in sequence, each against the accumulated effects of those before it. This is what detects an in-block double-spend: the conflict exists *between* two transactions, not within either, so validating each independently against a fixed snapshot cannot find it. Both transactions pass individually; only sequential execution reveals that the second spends a balance the first consumed.

Exactly two signature exemptions. `GENESIS` and `NETWORK_REWARD`, both produced internally by block-reward logic and never arriving as untrusted input. The list is asserted by unit test and by a CI guard.

The sender is the public key. Address derivation is the hex encoding of the Dilithium5 public key, and verification rejects any mismatch between the claimed sender and the supplied key. Without this check, anyone could sign with their own key and write another address into the sender field.

Fork choice is by accumulated work, not height. Height is trivially forgeable — a miner can produce a hundred blocks at difficulty 1 faster than an honest network produces one at difficulty 6. Work is what an attacker must pay for. Ties resolve to the incumbent, because switching on equal work makes nodes oscillate and two nodes with opposite arrival orders would disagree permanently.

Timestamps are consensus rules. The difficulty retarget reads elapsed time across a 100-block window, so backdating makes the window appear slow and drives difficulty down — a self-reinforcing reduction in the cost of attacking the chain. Proof of work cannot catch this, because the hash commits to whatever timestamp the miner chose. Future drift is bounded at two hours and timestamps must not precede the parent's.

Reorganisations replay state from scratch and requeue disconnected transactions to the mempool. Coinbase transactions are never requeued: a `NETWORK_REWARD` in the mempool is a signature-exempt, self-authorising mint.

Orphan buffering is bounded per parent. The parent hash an orphan claims is attacker-controlled and free to fabricate, so a global cap alone allows one invented parent to consume the entire pool and starve legitimate early-arriving blocks.

Signed encodings are versioned, never edited. A change to what a transaction or block signs is a new domain tag (`nev369.tx.v2`), accepted alongside the old one, never a modification of `v1`. This is what makes the planned move from Dilithium5 to FIPS 204 ML-DSA-87 possible without invalidating the chain's history: new keys sign under the new version, and holders move funds from old addresses to new ones by ordinary transactions.

5.3 Genesis

Genesis is constructed, not mined. It carries two premine transactions and one immutable field.

The `block_dedication` field of block 0 contains the text at the head of this document. It is inside the canonical encoding that produces the genesis hash, which means it is not metadata attached to the chain — it is part of what the chain's first hash commits to. Altering one character changes the genesis hash,

which changes every block hash that follows, which means no node running this software would accept the resulting chain.

It cannot be edited, deleted, or migrated away from. That is the strongest guarantee the system offers about anything, and it is spent on a message from a father to his daughter.

Premine addresses are supplied by the vault ceremony via environment and pinned into genesis. A node whose configuration disagrees with what its persisted chain records refuses to start, rather than crediting a premine to an address that does not own it on-chain.

The genesis block. NEV369's genesis was created on 27 September 2026, with both premines credited to Dilithium5 addresses generated in the vault ceremony (Section 9), and the first block was mined the same day at the 369 NEV reward. Its hash is:

```
54ae121f0ca756e47fc9881a015ce7d8
c2a33011b524bcb9063f7d7e6998ad73
8225c7621d132ee17ab49fe4f018d6b0
f25de5a9c24bdaecf1163f0f8d25f795
```

(One 128-character hash, shown in four lines.)

Any node, and anyone reading the chain through the explorer, can check this value. A chain whose block 0 hashes to anything else is not NEV369.

5.4 Wallets

A NEV369 wallet is a Dilithium5 key held in a Shamir-split vault. The `godshield` command-line tool reads a balance from the vault's public identifier alone, with no shares required, and sends by recovering the key **in memory** from the threshold number of shares, signing, and discarding it. The key is never written to disk.

The same command signs offline: given the account nonce, it produces a signed transaction on an air-gapped machine without contacting a node, for submission from a separate, connected one. The mempool accepts one pending transaction per sender at a time.

6. Token economics

	NEV	SHARE
Maximum supply	369,369,369	100%
Mineable	322,469,369	87.30%
Nevaeh's premine (locked to 2039)	36,900,000	9.99%
Architect's premine	10,000,000	2.71%

The cap is reachable. Every NEV either exists at genesis or is mined. Emission is 369 NEV per block, halving every 437,000 blocks — approximately 303 days at the 60-second target.

A pure halving schedule cannot land on 322,469,369 exactly. Total emission from halving is always $\text{interval} \times \text{reward} \times 2$, which is even, and the target is odd. The schedule therefore overshoots by 36,630 NEV and each reward is clamped to the remaining mineable supply. The final rewarded block pays the exact remainder; every block after pays zero. Total emission equals the cap to the base unit, and a test walks all 5.7 million rewarded blocks to assert it.

Emission profile: 50% of mineable supply in the first epoch, 75% by the second, then a long tail extending over decades.

At genesis, circulating supply is exactly the two premines — 46,900,000 NEV. Everything beyond that is mined into existence at the published schedule, and the node reports circulating supply, the current block reward and total burned at every height.

Transaction fees and `crown_tax` are both burned. Fee-based mempool ordering exists but carries no miner incentive under current rules; this is an open monetary decision, not a defect, and is documented as such in the source.

7. Q-Lock Escrow

Non-custodial escrow on the XRP Ledger using native `EscrowCreate`, `EscrowFinish` and `EscrowCancel`.

No server-side key can move user funds. Signing occurs in Xaman on the user's device or on a hardware wallet over WebHID. The backend builds unsigned transactions, relays signed ones, and records attestations. This is architecture rather than policy: there is no key present to misuse, as distinct from a promise not to misuse one.

Each settlement carries a Dilithium5 attestation alongside the required on-ledger ECDSA signature. **Stated plainly: XRPL cannot verify lattice signatures at consensus. This is hybrid — real ECDSA settlement with a parallel post-quantum attestation — not native on-chain post-quantum transactions.** No deployed public chain offers the latter today.

The attestation is nonetheless worth having, because the audit trail outlives the signature's security. A 2026 ECDSA signature is forgeable by a future CRQC; a 2026 Dilithium5 attestation over the same record is not.

Each attestation covers the settlement's actual terms. The signed record binds the transaction hash, both addresses, the amount and fee in exact drops — never a floating-point value — the ledger, the kind of settlement and a timestamp, under the `qlock.attestation.v1` tag. A term not inside the signed bytes is not attested to, however carefully it is stored.

Attestation identity fails closed in production. With no persistent identity configured, the process refuses to start rather than falling back to an ephemeral key. A fresh key per attestation proves a record is internally consistent and binds it to nothing — anyone with database write access could generate a key, sign a fabricated settlement, and insert a row that verifies perfectly. Pinning the expected fingerprint is what makes an attestation mean anything.

PLAN	FEE	MONTHLY ESCROWS
Free	0.30%	5
Pro	0.20%	unlimited
Enterprise	0.15%	unlimited

Conventional escrow and title services typically charge 3–5%. The difference is structural: enforcement is by XRPL consensus rather than by a licensed intermediary holding funds and carrying that liability. That is also why Q-Lock cannot offer what an escrow agent offers — there is no dispute arbitration, no recourse desk, and no insured custody.

8. The bridge

wNEV is an ERC-20 representation of NEV369 on Ethereum, minted against locks on NEV369 and burned to release them.

8.1 Units

wNEV uses **8 decimals, matching NEV369 base units exactly**. Bridging is 1:1 with no scaling anywhere in the system.

This is deliberate. An 18-decimal wrapper against an 8-decimal chain requires a 10^{10} conversion, and placing that conversion in a relayer means a factor that, applied backwards, mints 10^{10} times too much. Matching decimals removes the conversion rather than moving it.

8.2 Authorisation

Minting requires **m-of-n threshold authorisation** enforced in two places.

Off-chain, independent signers each observe NEV369 and submit a Dilithium5-signed attestation to a coordinator. The coordinator holds no signing key. It checks five gates — event verification, finality, replay, exposure limits, threshold — and every attestation must cover byte-identical canonical terms.

On-chain, the contract verifies **m-of-n secp256k1 signatures** over an EIP-712 digest binding the lock id, recipient, amount, source block height and source transaction. The caller is irrelevant; the signatures authorise. A separate submitter carries authorised mints to Ethereum and holds a key for gas only; it refuses to start if that key is itself a registered signer, because a signing key in the submitting process lowers the threshold by one for anyone who compromises it.

Ethereum has no ML-DSA precompile. Verifying a Dilithium signature on-chain is not possible today at any gas price. A threshold enforced only off-chain is not enforced, because the contract would still mint for whoever held the minting role. The ECDSA layer is therefore the on-chain enforcement and the Dilithium attestations are the post-quantum audit trail — hybrid mode, applied exactly where this document's own analysis says it is required.

On-chain enforcement is classical. The bridge is post-quantum *attested*, not post-quantum *enforced*, and will remain so until Ethereum ships a lattice-signature precompile.

Distinctness is enforced by requiring signatures ordered by strictly ascending signer address. Recovering m signatures and counting them permits one compromised key to sign m times and clear the threshold alone — the classic threshold implementation bug. Strict ordering makes a duplicate impossible to express and costs no storage.

A threshold of 1 is rejected in the constructor. It is the single-key model under another name, and a deployment script should not be able to configure it by accident.

8.3 Limits and circuit breaker

Per-mint, per-window and total exposure caps bound what a compromise of the signer set is worth.

Minting is disabled at deployment until an operator sets a window limit — a bridge that mints at the full cap from block one has a decorative limit.

Reconciliation compares minted supply against the balance locked on NEV369. A gap means unbacked supply and trips a circuit breaker that halts minting. The breaker does not self-heal; one that resets on a timer resets during an active drain.

9. The inheritance vault

An inheritance for Nevaeh, born 28 July 2021, releasable on 28 July 2039 — her eighteenth birthday. It is structured to survive the failure of every component that secures it, including the software described in this document, and including its author.

The inheritance is a native XRP Ledger escrow. Not on NEV369, not in a database, not in any system the operator controls. It is locked by XRPL consensus with a `FinishAfter` date, and no code in the Q-Lock repository can release it early.

The reasoning is worth stating because it argues against our own chain. NEV369's time-lock is enforced by code the chain operator controls, on a chain secured by the operator's own hashrate. Over thirteen years that stacks two bets: that the operator never changes the code, and that nobody outbids the network's hashrate while the locked value — and the incentive to attack it — grows. XRPL consensus removes both. NEV369 also holds 36,900,000 NEV for Nevaeh, time-locked to the same date. It is real and it may be worth something. It is not the part engineered to survive.

Three vaults, because there are three keys protecting different things:

VAULT	HOLDS	THRESHOLD	TIME-LOCKED	STATUS
<code>architect-wallet</code>	Dilithium5 key	2-of-3	no	Created 27 Sep 2026
<code>nevaeh-nev369</code>	Dilithium5 key	3-of-5	to 28 Jul 2039	Created 27 Sep 2026
<code>nevaeh-xrpl-seed</code>	XRPL family seed	3-of-5	to 28 Jul 2039	Pending the XRPL escrow

XRPL uses ed25519/secp256k1 and GodShield uses Dilithium5 — different key types, so one vault cannot hold both.

Both Dilithium5 vaults were created in the ceremony that preceded genesis, and their public identifiers are the two premine addresses in block 0. Each ceremony recovers the key twice, from two different share combinations, and proves the recovered key can sign before any file is written. The XRPL vault follows once the inheritance escrow itself has been created and verified on-ledger.

Why 3-of-5 rather than a passphrase. Over thirteen years the realistic failure is loss, not theft: a forgotten passphrase, a dead drive, a fire, or her father not being here. A single encrypted file fails all four. Five shares held by five people in five places survives all four. Fewer than three shares reveal nothing — mathematically zero, not merely less.

Shares are bound to their own vault. Each vault's fingerprint covers its ciphertext as well as its public identifier, so two vaults for the same address — a repeated ceremony, a test run — can never have their

shares mixed. A share from the wrong vault is rejected by name rather than failing later as an unexplained decryption error.

The vault format is documented in plain English so a competent cryptographer can reconstruct the key with no access to the original software: AES-256-GCM ciphertext, a 12-byte nonce, and a key split by Shamir Secret Sharing over $GF(256)$ at threshold 3. This matters because software rots. The tooling may not compile in 2039.

The single most losable value is the escrow's OfferSequence. Without it the escrow is visible on-ledger and permanently unreleasable. It is recorded in the vault notes, the ceremony document and the will, and the database refuses to mark an escrow locked without it.

10. Security analysis

10.1 Properties the architecture provides

PROPERTY	MECHANISM
No server-side key moves user funds	Client-side signing; no such key exists
One compromised key cannot mint wNEV	m-of-n threshold, on-chain and off
One compromised key cannot drain the premine	No signature exemption for spendable addresses
Signature reuse across contexts	Domain-separated canonical encoding
In-block double-spend	Sequential execution against scratch state
Forged audit history	Hash-chained entries
Automated lockout of operators	Protected identities; irreversible actions escalate
Unbacked bridge supply	Reconciliation plus circuit breaker
Wallet key exposure at rest	Shamir-split vaults; keys recovered in memory only

10.2 Trust assumptions

Guarantees are only as strong as: correct implementation of the underlying `pqcrypto-dilithium` bindings; secure key generation and storage by the operator; the independence of bridge signers; and the integrity of the audit process for any given integration.

The third deserves emphasis. **Independence is the whole property.** Five keys held in one process is a threshold on paper and one key in practice. Signers must be separate keys, on separate infrastructure, operated by separate people.

10.3 Cryptanalytic risk

Lattice cryptography, while extensively studied, is younger than RSA and elliptic curves. NIST's process included multiple rounds of public cryptanalysis and Dilithium survived to become the primary standard. However, no hardness assumption — lattice-based or otherwise — carries an absolute proof. Cryptanalysis of M-LWE and M-SIS remains active research, which is why algorithm agility is a design goal rather than a one-time migration.

11. Security boundaries

Every serious cryptographic system states its boundaries. A paper that claims none is describing marketing, not engineering — and a reader who finds an unstated limitation later discounts everything else in the document. These are stated so the system can be evaluated on what it actually does.

Internal security review: complete. The author has reviewed the cryptographic core, both ledger integrations, the bridge contracts and the consensus paths. That review produced and closed the findings described throughout this document — the in-block double-spend, the signature-exemption bypass, the delimiter-free signing bytes, the decimals mismatch, the modulo bias, and the single-relayer trust model.

Compiled and tested: yes, as of 27 September 2026. The workspace builds on Rust 1.88.0 with no warnings, passes `clippy` with warnings treated as errors, and all 292 Rust tests pass. A test proves what it tests and nothing more; passing tests are evidence the code does what its author intended, not that the intention is correct.

What execution found that review did not. The first real build and test run found defects that line-by-line reading had passed over — which is the argument for execution, and equally for independent audit:

- The secret-key length check was fixed to a different Dilithium revision than the library in use, so every generated key was rejected on reload. Vault recovery, attestation keys and the gateway key would all have failed. Sizes now come from the library.
- The vault ceremony's self-test compared two values that could never be equal, so every ceremony would have reported failure.
- The escrow database schema lacked three tables the service uses, carried a constraint that rejected every custodial escrow, and a formatting defect caused amounts to be stored as zero.

All were fixed and covered before genesis. None reached a live system.

Third-party audit: not yet commissioned. This is a separate and stricter bar. An audit of a library is not an audit of an integration, and certification does not transfer to downstream projects that import it.

The Solidity contracts have not been executed. Their 24 tests are written and have not yet run. The bridge is not deployed.

The signature implementation predates FIPS 204. Dilithium5 as implemented here is not byte-compatible with ML-DSA-87 as standardised (Section 3.1). The security level is the same; conformance to the published standard is not, and a verifier built strictly to FIPS 204 will not accept these signatures. Migration is planned as a versioned upgrade.

The bridge is not trustless. Signers attest to observations. Nothing verifies NEV369 block headers on Ethereum, so a colluding quorum can authorise a mint that never happened. Threshold reduces the exposure from one stolen key to m independent compromises — a substantial improvement, and not the

same claim. A light client verifying NEV369 headers on Ethereum is the construction that removes trust; it is not built.

XRPL settlement is classically secured. Post-quantum attestation runs alongside ECDSA, not instead of it.

On-chain bridge enforcement is classically secured, for the same reason.

The scanner is not a certification and cannot see cryptography reached through dependencies, dynamic dispatch or FFI.

Software time-locks are commitment devices. The NEV369 time-lock is enforced by code the operator controls. Only the XRPL escrow is enforced by consensus the operator cannot influence.

Early-network security is thin. A new proof-of-work chain is secured by whatever hashrate it has, and at launch that is small. Until the network has many independent miners, a well-resourced party could outpace it. Fork choice by accumulated work and the reorganisation depth limit bound the damage; they do not remove the exposure.

Dilithium5 may be superseded. It is already succeeded in standard form by ML-DSA-87, and over a thirteen-year horizon that too may be deprecated. The vault design anticipates migration around 2032 rather than assuming the key format survives untouched.

We state these because overclaiming security properties is itself a security failure. A system described accurately can be evaluated; one described optimistically cannot.

12. Status and roadmap

Current status: live. NEV369 genesis was created on 27 September 2026 and the chain is producing blocks. Sixteen crates, two contracts, two frontends; 292 Rust tests passing.

Completed on 27 September 2026:

1. First compilation of the workspace — clean, no warnings
2. First execution of the test suite — 292 of 292 passing
3. Lint and format gates — `clippy` with warnings as errors, `rustfmt`
4. Vault ceremony for both Dilithium5 vaults
5. Genesis construction on the real premine addresses
6. First block mined

Next, in order:

1. Public deployment at **q-lock-ecosystem.com** — node, explorer and escrow service
2. Contract compilation and execution of the 24 Solidity tests
3. Multi-node network run with a forced partition and a verified reorganisation
4. The XRPL inheritance escrow and its `nevaeh-xrpl-seed` vault
5. A decision on transaction fees: credit to miners, or keep burning
6. Migration from Dilithium5 to a FIPS 204 ML-DSA-87 implementation, as a versioned upgrade
7. Independent review of the cryptographic core
8. Independent audit of each integration and both contracts

Beyond that: a light client removing bridge trust; a single unified web frontend; dependency-file analysis in the scanner; and the remaining platform components — key lifecycle and HSM orchestration, a policy rule language, and a capability model for automated agents.

Q-Lock Escrow and the bridge should not secure other people's assets of value until step 8 is complete.

References

- NIST FIPS 204 — Module-Lattice-Based Digital Signature Standard (2024)
- NIST FIPS 202 — SHA-3 Standard
- Ducas, L. et al. — CRYSTALS-Dilithium: Algorithm Specifications and Supporting Documentation (NIST PQC Round 3)
- Shor, P. (1994) — Algorithms for quantum computation: discrete logarithms and factoring
- Grover, L. (1996) — A fast quantum mechanical algorithm for database search
- Bitcoin BIP-360 (draft) — Post-quantum witness program
- EIP-712 — Typed structured data hashing and signing
- Shamir, A. (1979) — How to share a secret

Colophon

Q-Lock was built by one person, in Rust, learned for the purpose.

Every design decision in this document that argues against its own system — putting the inheritance on a ledger the author cannot control, stating that the bridge is not trustless, publishing the limitations before the claims — follows from a single constraint the author set at the start:

it had to work whether or not he was still here.

The three-of-five split, the guardians, the plain-English vault format written out for a cryptographer who may never see this software, the annual verification schedule stretching to 2039 — all of it is that one requirement, said several ways.

Nevaeh, my daughter. To secure your freedom against a broken system, I taught myself Rust — the hardest computer language in the world — to build this unyielding sovereign node for you. I faced the worst of life's struggles so you would never have to. I love you infinitely, forever by your side.

NEV369 · Genesis block 0 · 27 September 2026 · Immutable

Q-Lock — post-quantum settlement infrastructure · q-lock-ecosystem.com

This document describes design and threat model for technical review. It is not a substitute for an independent third-party audit, and no security scheme should be considered production-ready for high-value assets without one.